

LaTeX のコンパイルと表示例

環境に依存しない理解と実践

1MDA1103

安 炯奎

2026 年 7 月 18 日

目次

1	LaTeX とは何か	3
2	LaTeX 文書は基本的にどのようにコンパイルされるか	3
2.1	最も単純な流れ	3
2.2	コンパイル中に行われていること	4
2.3	なぜ複数回コンパイルすることがあるのか	4
3	代表的なコンパイル方式と使い分け	5
3.1	方式の比較	5
3.2	選び方の目安	5
4	コンパイルに関係する代表的なプログラム	6
5	異なる環境で同じ結果を得るための設定	7
5.1	実行環境よりも、ビルド条件を揃えることが重要である	7
5.2	コンパイラ、ドライバ、補助プログラムの関係	7
5.3	手動実行と <code>latexmkrc</code> による自動化	8
5.4	Overleaf で追加設定が必要になる場合	9
5.5	設定が一致しないと大量のエラーになる理由	9
5.6	環境を移すときに追加確認すべきこと	9
6	代表的な LaTeX 作成環境と固有の特徴	10
7	表示例から作ってみる LaTeX	11
7.1	タイトル、目次、見出しを作る	11
7.2	文字の見た目を変える	11
7.3	段落と三種類のリストを作る	12
7.4	数式を作る	12
7.5	番号を付けて別の場所から参照する	12
7.6	図を配置する	12
7.7	表を作る	13
7.8	ソースコード、URL、脚注を表示する	13

8	少し特殊な機能の記述例	14
8.1	参考文献と <code>references.bib</code>	14
8.2	TikZ を使って LaTeX 内で図を描く	15
8.3	二段組でページ全体の幅を使う図	16
8.4	外部ソースコードを読み込む	16
8.5	独自コマンドを定義する	16
8.6	長い文書を複数ファイルへ分割する	16
9	pLaTeX 向けの最小テンプレート	17
10	エラーを読むときの基本方針	17

1 LaTeX とは何か

LaTeX を理解する際は、**TeX**、**LaTeX**、**コンパイラ**、**TeX ディストリビューション**を区別すると分かりやすい。

TeX	Donald Knuth によって開発された組版システムであり、文字、数式、行、ページを配置する基礎的な処理を担当する。
LaTeX	TeX の上に構築された文書作成用の仕組みである。利用者は文字の座標を直接指定するのではなく、 <code>section</code> 、 <code>figure</code> 、 <code>equation</code> などの論理的な命令を記述する。
TeX エンジン	pdfTeX, XeTeX, LuaTeX, pTeX, upTeX など、実際に入力を読み、組版処理を実行するプログラムである。
LaTeX コンパイラ	日常的には、LaTeX 形式を読み込んだエンジンを実行するコマンドを指す。代表例は <code>pdflatex</code> 、 <code>xelatex</code> 、 <code>lualatex</code> 、 <code>platex</code> である。
TeX ディストリビューション	エンジン、LaTeX 本体、パッケージ、フォント、BibTeX などをまとめた配布物である。代表例は TeX Live、MiKTeX、MacTeX である。

したがって、LaTeX は単独のアプリケーション名というより、複数のエンジン、形式、パッケージ、補助プログラムから構成される文書作成環境である。LaTeX 本体や対応エンジンの更新情報は、LaTeX Project の公開資料から確認できる [1]。

2 LaTeX 文書は基本的にどのようにコンパイルされるか

2.1 最も単純な流れ

LaTeX 文書は、通常、次の流れで処理される。

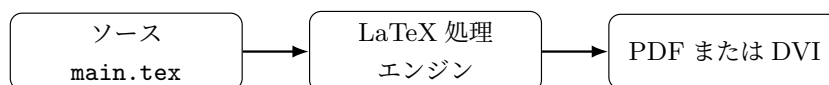


図 1: LaTeX コンパイルの基本的な流れ

例えば、pdfLaTeX では次のコマンドを実行する。

```
1 pdflatex main.tex
```

処理に成功すると、通常は `main.pdf` が生成される。一方、伝統的な LaTeX や pLaTeX では、まず DVI を生成し、その後 PDF へ変換する。

```
1 platex main.tex
2 dvipdfmx main.dvi
```

2.2 コンパイル中に行われていること

大まかには、次の処理が順番に行われる。

1. `main.tex` を読み込む。
2. `documentclass` で指定された文書クラスを読み込む。
3. `usepackage` で指定されたパッケージを読み込む。
4. コマンドやマクロを展開する。
5. 段落、数式、図表、ページを組版する。
6. PDF または DVI を出力する。
7. 参照番号などを `.aux`、目次を `.toc` などの補助ファイルへ保存する。

2.3 なぜ複数回コンパイルすることがあるのか

LaTeX は、文書の前方に書かれた参照から、後方にある図や節の番号を一度で知ることができない。そのため、最初の実行で番号を補助ファイルに保存し、次の実行でその情報を読み直す。

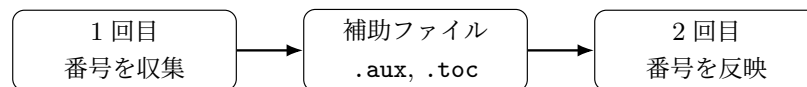


図 2: 相互参照を確定するための複数回コンパイル

参考文献を BibTeX で処理する典型的な順序は次のとおりである。

```
1 platex main.tex
2 pbibtex main
3 platex main.tex
4 platex main.tex
5 dvipdfmx main.dvi
```

ただし、通常は `latexmk` が必要な回数と順序を自動判定するため、利用者が毎回すべてのコマンドを手動実行する必要はない。

3 代表的なコンパイル方式と使い分け

コンパイラの選択では、文書クラス、文字コード、フォント、出力経路、投稿先の指定を確認する [2, 1].

3.1 方式の比較

表 1: 代表的な LaTeX コンパイル方式

コマンド	主な出力	長所	主な用途・注意点
<code>latex</code>	DVI	古典的で単純な DVI 経路を利用できる	EPS や古い DVI ワークフロー, 古いテンプレート. 通常の新規文書では使用機会が少ない
<code>pdflatex</code>	PDF	高速で安定し, 既存パッケージや投稿テンプレートとの互換性が高い	欧文中心の一般文書, 出版社指定テンプレート. システムフォントや Unicode の扱いには制約がある
<code>xelatex</code>	PDF	UTF-8 と OpenType/TrueType フォントを扱いやすい	多言語文書, 任意のシステムフォントを使う文書. <code>fontspec</code> を利用する場合
<code>lualatex</code>	PDF	Unicode, 現代的フォント, Lua による拡張に対応する	新規の多言語文書, 高度なフォント処理, Lua を使った自動化や拡張
<code>platex</code>	DVI	日本語組版の長い実績があり, <code>jsarticle</code> 等との互換性が高い	日本の大学・学会・出版社が pLaTeX を指定する場合. 通常は <code>dvipdfmx</code> で PDF 化する
<code>uplatex</code>	DVI	pLaTeX 系の日本語組版を保ちながら Unicode 範囲を扱いやすい	日本語中心で, より広い文字集合を扱う文書. テンプレート側の対応確認が必要

3.2 選び方の目安

- 投稿先や大学からコンパイラを指定されている場合は, その指定を最優先する.
- 欧文中心で, 既存テンプレートとの互換性を重視するなら pdfLaTeX が無難である.
- Unicode, 多言語, OpenType フォントを簡単に扱うなら XeLaTeX または LuaLaTeX を使う.
- 新規の Unicode 文書で拡張性を重視するなら LuaLaTeX が有力である.
- `jsarticle`, `jsbook` などを使う既存の日本語資料では pLaTeX または upLaTeX を検討する.
- 古い DVI, EPS, PSTricks 中心の資産を維持する場合は LaTeX 経路が必要になることがある.

4 コンパイルに関する代表的なプログラム

「LaTeX をコンパイルするもの」には複数の種類がある。

表 2: LaTeX 環境を構成する代表的なプログラム

分類	代表例	役割
TeX エンジン	pdfTeX, XeTeX, LuaTeX, pTeX, upTeX	マクロ展開, 段落分割, 数式やページの組版を実行する
LaTeX 実行コマンド	pdflatex, xelatex, lualatex, platex, uplatex	各エンジンに LaTeX 形式を読み込ませて文書进行处理する
DVI ドライバ	dvipdfmx, dvips	DVI を PDF または PostScript へ変換する
自動ビルド	latexmk, arara	必要なコンパイル回数, BibTeX, 索引処理などを自動化する
参考文献処理	bibtex, pbibtex, upbibtex, biber	.bib から参考文献一覧を生成する
索引処理	makeindex, mendex	索引語を並べ替え, 索引ファイルを生成する
ディストリビューション	TeX Live, MiKTeX, MacTeX	エンジン, パッケージ, フォント, 補助ツールをまとめて提供する
エディタ・実行環境	TeXstudio, TeXworks, VS Code, Overleaf 等	ソース編集, コンパイル操作, PDF 表示, エラー確認を支援する

重要なのは、**エディタとコンパイラは同じものではない**という点である。例えば VS Code は文章を編集するためのエディタであり、実際のコンパイルにはローカルへインストールした TeX Live などを利用する。Overleaf はエディタとサーバ上の TeX Live を一体として提供する。

5 異なる環境で同じ結果を得るための設定

5.1 実行環境よりも、ビルド条件を揃えることが重要である

LaTeX 文書は、ローカル PC、大学の演習室、CI サーバ、クラウドサービスなど、さまざまな環境で処理できる。ただし、同じ `.tex` ファイルを置くだけで、必ず同じ結果になるとは限らない。再現性を高めるには、少なくとも次の条件を揃える。

- 使用するコンパイラと処理経路：例として `platex -> dvipdfmx`
- TeX Live などのディストリビューションと、可能であればその版
- 文書クラス、パッケージ、スタイルファイル
- 使用するフォント
- 画像、`.bib`、分割した `.tex` などの外部ファイル
- BibTeX や索引処理を含むビルド順序

重要なのは、特定のエディタやサービスを使うことではなく、**どのプログラムを、どの順番で、どの設定によって実行するか**を共有することである。

5.2 コンパイラ、ドライバ、補助プログラムの関係

コンパイラと画像・ハイパーリンク用ドライバの指定は一致させる必要がある。例えば、次の指定は DVI を経由する処理方式を表している。

```
1 \usepackage[dvipdfmx]{graphicx}
2 \usepackage[dvipdfmx]{xcolor}
3 \usepackage[dvipdfmx]{hyperref}
```

これは `pLaTeX -> DVI -> dvipdfmx -> PDF` という経路を前提とする。このコードを `pdfLaTeX` で処理すると、`pdfLaTeX` は DVI を使わないため、`Wrong DVI mode driver option` などのエラーが発生することがある。`pdfLaTeX`、`XeLaTeX`、`LuaLaTeX` 向けの新規文書では、通常はドライバを明示せず、パッケージの自動判定に任せる方が簡単である。

`pLaTeX` 文書で参考文献と索引も作成する場合、処理の関係は図 3 のようになる。`pbibtex` と `mendex` は常に必要なわけではなく、参考文献または索引が存在する場合だけ、最初の `platex` の後に実行される。

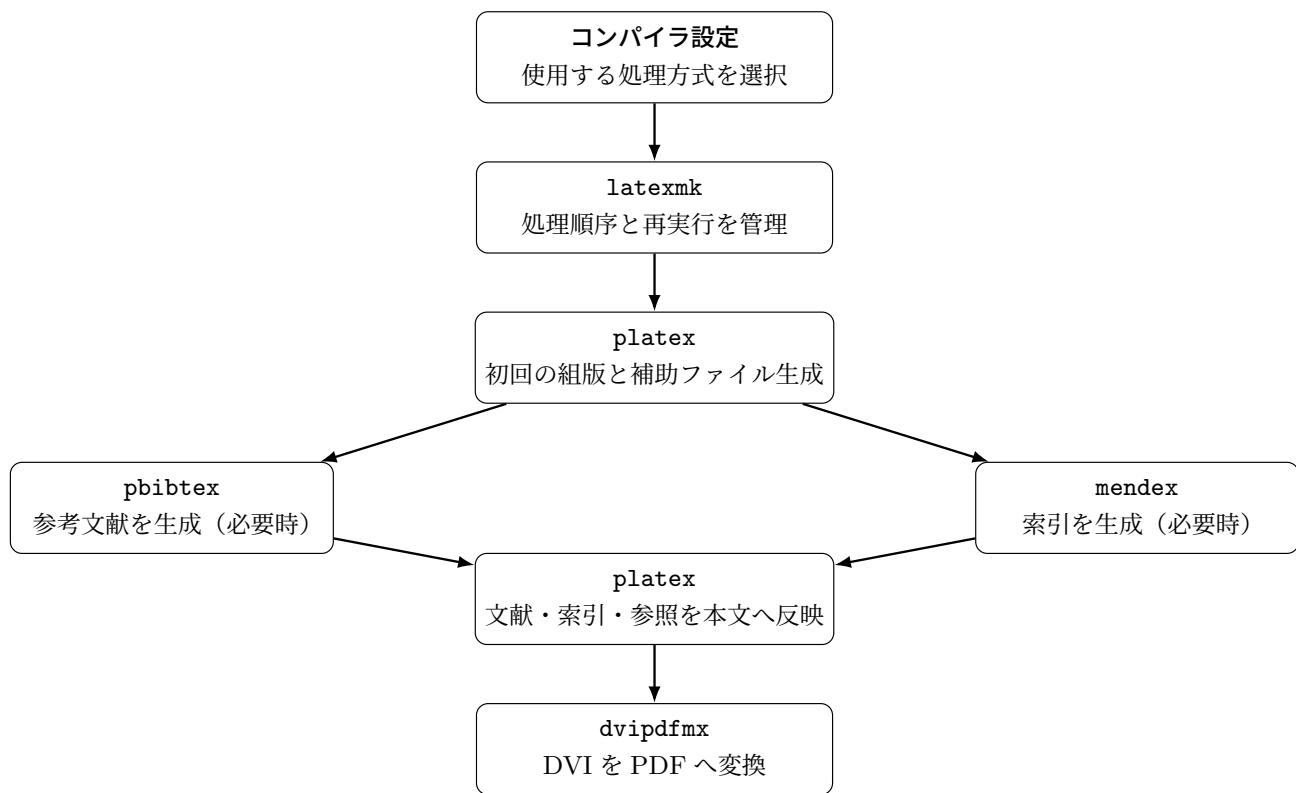


図 3: pLaTeX 文書を PDF にする際の代表的な実行順序

5.3 手動実行と latexmkrc による自動化

参考文献と索引を含む文書を手動で処理する場合、代表的には次のように実行する。参考文献がなければ pbibtex を、索引がなければ mendex を省略する。参照番号が確定するまで、platex をさらに実行することもある。

```

1 platex main.tex
2 pbibtex main
3 mendex main.idx
4 platex main.tex
5 platex main.tex
6 dvipdfmx main.dvi

```

毎回この順序を手で管理する代わりに、プロジェクト直下へ latexmkrc を置いて、使用するプログラムと PDF 生成方式を指定できる。

```

1 $pdf_mode = 3;
2 $latex = 'platex';
3 $bibtex = 'pbibtex';
4 $dvi2pdf = 'dvipdfmx %0 -o %D %S';
5 $makeindex = 'mendex %0 -o %D %S';

```

この設定があれば、利用者は次の一つのコマンドから処理を開始できる。latexmk は補助ファイルの変化を確認し、必要なプログラムと再実行回数を判断する。

```

1 latexmk main.tex

```

この方法はエディタに依存しない。TeXstudio, VS Code, JetBrains 系 IDE, コマンドライン, CI などから同じ latexmk 設定を呼び出せば、ビルド条件を共有しやすくなる。

5.4 Overleaf で追加設定が必要になる場合

Overleaf のコンパイラ選択欄には、通常、pdfLaTeX, LaTeX, XeLaTeX, LuaLaTeX が表示されるが、pLaTeX と upLaTeX は直接表示されない。そのため、pLaTeX 文書では Compiler を **LaTeX** に設定し、前節と同じ latexmkrc をプロジェクト直下へ置いて、内部で使用するプログラムを指定する [3, 4]。

これは文書を Overleaf 専用書き換える操作ではない。他の環境ではコマンドやエディタ設定から指定できるビルド条件を、Overleaf の UI 上でも同じになるように補う操作である。

5.5 設定が一致しないと大量のエラーになる理由

例えば、jsarticle を使用し、graphicx, xcolor, hyperref に dvipdfmx を指定した文書は、pLaTeX 系の処理を前提とする。これを pdfLaTeX で処理すると、文書クラスやドライバの不一致が最初に発生し、その後のパッケージで Undefined control sequence などの連鎖エラーが表示されることがある。この場合、個々のエラーを順番に直すのではなく、最初にコンパイラと処理経路を確認する。

5.6 環境を移すときに追加確認すべきこと

- ファイル名の大文字と小文字を区別する環境がある。
- Main document またはルート文書として指定された .tex ファイルが正しいか確認する。
- TeX Live の版により、パッケージの版や挙動が変わる場合がある。
- ある PC にだけ存在するフォント、クラス、スタイルファイルは他の環境では利用できない。
- 特殊なビルド手順は latexmkrc, Makefile, エディタ設定などで明示する。

6 代表的な LaTeX 作成環境と固有の特徴

LaTeX 文書は、専用エディタ、汎用 IDE、ブラウザ、コマンドラインなどから作成できる。ソース編集、コンパイル、PDF 確認といった多くの環境に共通する機能はここでは繰り返さず、各環境を選ぶ理由になりやすい固有の特徴だけを示す。

表 3: 代表的な LaTeX 作成環境の固有の特徴

環境	その環境に特徴的な点	適した利用場面
Cloud LaTeX	日本語 LaTeX への対応を前面に出した国内サービスであり、日本語文書を作るための初期設定を省きやすい。科研費申請書、日本の学会・論文誌、和文書籍など、日本語圏で利用される形式のテンプレートが充実している [?].	日本語文書をすぐに作り始めたい場合、日本国内向けの申請書・論文・書籍テンプレートを利用したい場合。
Overleaf	複数人の同時編集、コメント、変更履歴など、共同執筆の機能が特に充実している。出版社・学会が公開する多数の投稿テンプレートや、GitHub, Dropbox, Zotero, Mendeley 等との連携も特徴である [5, 6].	共著者とリアルタイムで執筆・査読する場合、海外誌を含む投稿テンプレートや外部サービス連携を重視する場合。
VS Code + LaTeX Workshop	ビルド手順を「recipe」として細かく定義でき、WSL, Docker, Remote Development と組み合わせやすい。プログラム、データ、LaTeX を同じワークスペースで扱えることも特徴である [?].	研究用コードと論文を同じリポジトリで管理する場合、遠隔・コンテナ環境を使う場合。
JetBrains 系 IDE + TeXiFy-IDEA	IntelliJ Platform のプロジェクト解析、検査、宣言への移動、構造表示を LaTeX と BibTeX にも適用できる。CLion, PyCharm, IntelliJ IDEA など普段の IDE 内に文書作成を統合できる [?].	JetBrains 系 IDE を普段から使い、ソースコードと論文の移動を減らしたい場合。
TeXworks	機能を意図的に絞った簡潔な画面を持ち、TeX への入口を低くすることを目的としている。Windows 版 TeX Live や MiKTeX に同梱される構成もあり、導入直後から基本的な流れを確認しやすい [?].	初めてローカル環境で LaTeX を試す場合、複雑な IDE 設定を避けたい場合。
TeXstudio	表・画像・数式を作るためのアシスタント、参照や文法の詳細な検査、複数のコンパイラや索引・参考文献ツールを組み合わせられる高度なビルド設定が強みである [?].	LaTeX 専用の機能を一つのローカルアプリケーションへ集約したい場合。

環境	その環境に特徴的な点	適した利用場面
コマンドライン + 任意のエディタ	Makefile, latexmkrc, シェルスクリプト, CI, コンテナを使って, 編集画面から独立した再現可能なビルドを構成できる. 特定のエディタへ処理手順を閉じ込めない.	自動テスト, 大規模文書, Git 中心の運用, 独自ツールとの連携を重視する場合.

Cloud LaTeX や Overleaf などのクラウド環境には、インターネット接続、コンパイル時間、サーバに存在しないフォントやツールを自由に追加しにくいという制約がある。一方、ローカル環境にはインストールと更新管理が必要である。どちらか一方だけを選ぶ必要はなく、ローカル環境で編集・自動化しながら、日本語向けテンプレートの利用には Cloud LaTeX、共同作業や投稿には Overleaf を使うなど、目的に応じて併用できる。

7 表示例から作ってみる LaTeX

この章では LaTeX の命令を先に提示しない。まず表示例を観察し、同じ結果を得るために必要な文法、環境、パッケージを自分で調べて作成する。検索する際は、「LaTeX 見出し」「LaTeX 行列」「LaTeX 図 挿入」のように、作りたい要素の名前を組み合わせるとよい。

7.1 タイトル，目次，見出しを作る

次のように、文書タイトル、著者、日付、目次、番号付きの見出しを持つ文書を作成してみよう。番号を付けない見出しも一つ作り、目次へ入れる場合と入れない場合の違いを確認する。

サンプルレポート	
山田太郎	
2026 年 7 月 18 日	
目次	
1 はじめに	1
1.1 背景	1
2 実験	2
1 はじめに	
1.1 背景	
1.1.1 詳細	

7.2 文字の見た目を変える

次の表示を一つの段落内に再現してみよう。どの命令が太字、斜体、下線、等幅文字、文脈に応じた強調を表すかを調べる。

太字，イタリック，下線，等幅文字，強調

7.3 段落と三種類のリストを作る

空行による段落分けと、記号付き、番号付き、語句説明付きの三種類のリストを作成してみよう。リストを入子にした場合の表示も確認する。

これは第 1 段落である。ソース上の単なる改行と、段落を分ける空行の違いを観察する。

これは第 2 段落である。

- 記号付き項目 A
 - 記号付き項目 B
1. 最初の手順
 2. 次の手順

CPU 中央処理装置

RAM 主記憶として使われるメモリ

7.4 数式を作る

文中の数式、中央に独立して置かれる数式、番号付き数式、複数行で等号を揃えた数式、行列をそれぞれ作成してみよう。必要になる数式用パッケージも調べる。

文中では、円の面積を $S = \pi r^2$ のように表示できる。

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$
$$\int_0^1 x^2 dx = \frac{1}{3} \tag{1}$$

$$(a+b)^2 = a^2 + 2ab + b^2, \tag{2}$$

$$(a-b)^2 = a^2 - 2ab + b^2. \tag{3}$$

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

7.5 番号を付けて別の場所から参照する

節、数式、図、表に番号を付け、本文からその番号を参照してみよう。参照先の前後を移動しても番号が自動更新されることを確認する。例えば、この文は式 1 を参照している。

7.6 図を配置する

次のように、図を中央へ配置し、説明文と図番号を付け、本文から参照できる形を作成してみよう。外部の PNG, JPEG, PDF 画像を一つ用意し、幅と配置候補を変えて結果を比較する。



図 4: 図の表示例

図 4 のような番号付き図を、自分で用意した画像でも作成してみよう。

7.7 表を作る

列の左揃え，中央揃え，右揃えを使い分け，すべての行を横線で区切った表を作成してみよう．列数，行数，数値を変更し，表番号と説明文も付ける．

表 4: 表の表示例

手法	正解率	時間 [s]
A	0.91	12.4
B	0.94	15.8

本文から表 4 を参照し，行を追加した後も番号が保たれることを確認する．

7.8 ソースコード，URL，脚注を表示する

次のようなソースコード枠，クリックできる URL，ページ下部の脚注を作成してみよう．ソースコードでは行番号，言語名，枠，改行方法などを変更して比較する．

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Hello, LaTeX!\n");
6     return 0;
7 }
```

LaTeX Project の Web サイトは <https://www.latex-project.org/> である．本文には補足用の脚注を置くこともできる^{*1}．

^{*1} 脚注は通常，ページ下部へ自動配置される．

8 少し特殊な機能の記述例

基本的な表示は前章の例を手掛かりに自分で調べることができる。一方、複数ファイルを連携させる機能や、文書全体の構成に関係する機能は、最初の形を知っていた方が調査しやすい。ここでは代表的な記述例を示す。

8.1 参考文献と references.bib

参考文献を本文へ直接すべて書く代わりに、書誌情報を references.bib へ分離して管理できる。ファイル名は自由だが、ここでは役割が分かりやすいように references.bib とする。同じ書誌データを複数の論文やレポートから再利用でき、本文側では引用キーだけを指定すればよい。

8.1.1 references.bib の記述例

```
1 @article{sample2026,
2   author = {Taro Yamada},
3   title = {A Sample Article},
4   journal = {Journal of Examples},
5   year = {2026},
6   volume = {1},
7   number = {1},
8   pages = {1--10}
9 }
10
11 @book{latexbook,
12   author = {Hanako Suzuki},
13   title = {Introduction to LaTeX},
14   publisher = {Example Press},
15   year = {2025}
16 }
```

表 5: BibTeX エントリの主な構成要素

要素	例	役割
エントリ種別	@article, @book, @inproceedings	文献の種類を表す
引用キー	sample2026	本文の cite{sample2026} と対応する識別子
フィールド	author, title, year 等	著者名, 題名, 発行年などの書誌情報を格納する
波括弧または引用符	title = {...}	各フィールドの値を囲む

本文では次のように引用キーを指定し、文書末尾で参考文献一覧の形式とファイル名を指定する。

```
1 Previous work proposed this method~\cite{sample2026}.
2
3 \bibliographystyle{ieeetr}
4 \bibliography{references}
```

`bibliography{references}` は、拡張子を省略して `references.bib` を指定している。 `bibliographystyle` は参考文献一覧の表示形式を指定する。 pLaTeX では `pbibtex`, upLaTeX では `upbibtex` を使う構成が多い。 近年の文書では、 `biblatex` と `biber` を使う方式もある。 利用するテンプレートや投稿先の指定がある場合は、その方式を優先する。

8.1.2 ファイル配置例

```
1 project/
2 |-- main.tex
3 |-- references.bib
4 |-- latexmkrc
5 |-- figures/
6 | '-- sample.png
7 |-- src/
8 | '-- main.py
9 '-- chapters/
10   |-- introduction.tex
11   '-- method.tex
```

本資料そのものも外部の `references.bib` を読み込み、末尾の参考文献一覧を生成している。

8.2 TikZ を使って LaTeX 内で図を描く

第 7 章の「入力 → 処理 → 出力」の図は、外部画像を読み込んだものではなく、TikZ パッケージを使って LaTeX のソース内で描画している。単純なフローチャート、ノードと矢印、幾何図形などは、画像編集ソフトを使わずに作成できる。

まず、プリアンプルでパッケージと必要なライブラリを読み込む。

```
1 \usepackage{tikz}
2 \usetikzlibrary{arrows.meta,positioning}
```

次のコードで、第 7.6 節と同じ構造の図を描画できる。 `node distance` は要素間の距離、 `right=of ...` は配置、 `draw` は枠線、 `fill` は背景色、 `draw[-{Latex}]` は矢印を表す。

```
1 \begin{figure}[tbp]
2   \centering
3   \begin{tikzpicture}[node distance=20mm]
4     \node[draw, rounded corners, fill=blue!8]
5       (input) 入力{};
6
7     \node[draw, rounded corners, fill=green!8,
8       right=of input]
9       (process) 処理{};
10
11    \node[draw, rounded corners, fill=orange!12,
12      right=of process]
13      (output) 出力{};
14
15    \draw[-{Latex}, thick] (input) -- (process);
16    \draw[-{Latex}, thick] (process) -- (output);
17  \end{tikzpicture}
18  \caption{で作成した処理の流れ TikZ}
19  \label{fig:tikz-flow}
20 \end{figure}
```

ノードの識別名である `input`, `process`, `output` を使って, 配置関係や矢印の始点・終点を指定している. 図の形が複雑になる場合は, TikZ のライブラリを追加したり, 図だけを別の `.tex` ファイルへ分離したりできる.

8.3 二段組でページ全体の幅を使う図

二段組文書で両方の段をまたぐ図には, 通常の `figure` ではなく `figure*` を使う.

```
1 \begin{figure*}[t]
2   \centering
3   \includegraphics[width=0.8\textwidth]{figures/wide.pdf}
4   \caption{A wide figure in a two-column document}
5   \label{fig:wide}
6 \end{figure*}
```

8.4 外部ソースコードを読み込む

長いプログラムを LaTeX 本文へコピーせず, 別ファイルから読み込むことができる.

```
1 \lstinputlisting[language=Python]{src/main.py}
```

8.5 独自コマンドを定義する

文書内で何度も使う表記は独自コマンドにすると, 表記を統一し, 後から一括変更しやすい.

```
1 \newcommand{\vect}[1]{\bm{#1}}
2 \newcommand{\R}{\mathbb{R}}
3
4 $\vect{x}$ \in \R^n$
```

8.6 長い文書を複数ファイルへ分割する

章ごとにファイルを分け, メイン文書から読み込む構成にできる.

```
1 % main.tex
2 \begin{document}
3 \input{chapters/introduction}
4 \input{chapters/method}
5 \input{chapters/experiment}
6 \end{document}
```

`input` は指定したファイルの内容をその位置へ読み込む. 大規模文書では `include` も使われる.

9 pLaTeX 向けの最小テンプレート

次のコードは、pLaTeX と dvipdfmx で日本語文書を作成するための最小例である。外部画像や数式用パッケージを使用しないため、main.tex と references.bib を同じフォルダへ置けば試することができる。ここではタイトル、目次、見出し、本文中の引用、参考文献一覧だけを示す。その他の機能は、第 7 章の表示例と第 8 章の記述例を手掛かりに各自で調べて追加する。

```
1 \documentclass[a4paper,11pt]{jsarticle}
2 \usepackage{url}
3
4 \title{LaTeX Sample}
5 \author{Your Name}
6 \date{\today}
7
8 \begin{document}
9
10 \maketitle
11 \tableofcontents
12
13 \section{Introduction}
14 This is a minimal pLaTeX document.
15 For information about LaTeX engines, see~\cite{latex-project-engines}.
16
17 \bibliographystyle{ieeetr}
18 \bibliography{references}
19
20 \end{document}
```

この例では、本文中の cite と文末の bibliography が references.bib の引用キー latex-project-engines に対応している。参考文献を含むため、処理には第 5 章で示した pbibtex が必要である。通常は latexmk を使えば、必要な再実行も含めて自動処理できる。

10 エラーを読むときの基本方針

1. ログの最初に出たエラーから確認する。
2. 文書クラスとコンパイラが一致しているか確認する。
3. dvipdfmx などのドライバ指定が処理方式と一致しているか確認する。
4. 括弧、環境の開始と終了、コマンド名のスペルを確認する。
5. ファイル名とパス、大文字・小文字を確認する。
6. 一度に多数のエラーが出た場合、最初の一件が原因の連鎖エラーでないか疑う。
7. 問題箇所を一時的にコメントアウトし、最小構成まで減らして原因を切り分ける。

参考文献

- [1] The LaTeX Project, “Latex news: On engine support,” 2024. <https://www.latex-project.org/news/2024/11/01/issue40-of-latex2e-released/>.
- [2] Overleaf, “Choosing a latex compiler.” Overleaf Learn, 2026. https://www.overleaf.com/learn/latex/Choosing_a_LaTeX_Compiler.
- [3] Overleaf, “Japanese: Using platex on overleaf.” Overleaf Learn, 2026. <https://www.overleaf.com/learn/latex/Japanese>.
- [4] Overleaf Documentation, “The latexmkrc file,” 2026. <https://docs.overleaf.com/managing-projects-and-files/the-latexmkrc-file>.
- [5] Overleaf, “Latex editor features and benefits,” 2026. <https://www.overleaf.com/about/features-overview>.
- [6] Overleaf Documentation, “Collaborating in overleaf,” 2026. <https://docs.overleaf.com/collaborating/collaborating-in-overleaf>.